

연산자 오버로딩 보고서

1. 이론 조사

연산자 오버로딩은 사용자가 정의한 타입도 기본 타입처럼 자연스럽게 연산할 수 있게 만드는 기능이다. C++은 연산자를 실제 함수 호출로 해석하며, 멤버 함수 또는 비멤버 함수 형태로 오버로딩한다. Python은 `__add__`, `__eq__` 같은 특수 메서드(dunder method)를 통해 연산자의 동작을 정의한다.

두 언어 모두 “객체에 자연스러운 연산 의미를 부여한다”는 공통점이 있지만, C++은 컴파일 시점의 타입, 참조, 값 범주, 암시적 변환을 강하게 고려하고, Python은 런타임의 동적 디스패치와 프로토콜 중심 설계를 따른다.

1-1. 산술 연산자

연산	C++	Python
+	<code>operator+</code>	<code>__add__</code>
-	<code>operator-</code>	<code>__sub__</code>
*	<code>operator*</code>	<code>__mul__</code>
/	<code>operator/</code>	<code>__truediv__</code>
%	<code>operator%</code>	<code>__mod__</code>

C++에서 산술 연산자는 멤버 함수 또는 비멤버 함수로 구현할 수 있다.

```
// 방법 1. 멤버 연산자
// z1 + z2 -> z1.operator+(z2)
class ComplexMember {
private:
    double re_; // 실수부
    double im_; // 허수부

public:
    ComplexMember(double re, double im)
        : re_(re), im_(im) {}

    ComplexMember operator+(const ComplexMember& rhs) const
    {
```

```

        return ComplexMember(
            re_ + rhs.re_, // 실수부끼리 더함
            im_ + rhs.im_  // 허수부끼리 더함
        );
    }
};

void memberExample() {
    ComplexMember z1(1.0, 2.0); // 1 + 2i
    ComplexMember z2(3.0, 4.0); // 3 + 4i

    ComplexMember z3 = z1 + z2;           // 실제로는 z1.operator+(z2)
    ComplexMember z4 = z1.operator+(z2); // 위 코드와 같은 의미

    // ComplexMember z5 = 1.0 + z1;
    // 오류: 왼쪽 피연산자 1.0은 double이어서
    //      1.0.operator+(z1)를 호출할 수 없다.
}

// 방법 2. 비멤버 연산자
// z1 + z2 -> operator+(z1, z2)
// 1.0 + z1 -> operator+(1.0, z1)
class ComplexNonMember {
private:
    double re_; // 실수부
    double im_; // 허수부

public:
    ComplexNonMember(double re, double im = 0.0)
        : re_(re), im_(im) {}

    friend ComplexNonMember operator+(
        const ComplexNonMember& lhs,
        const ComplexNonMember& rhs
    );
};

```

```

    friend ComplexNonMember operator+(
        double lhs,
        const ComplexNonMember& rhs
    );
};

ComplexNonMember operator+(
    const ComplexNonMember& lhs,
    const ComplexNonMember& rhs
) {
    return ComplexNonMember(
        lhs.re_ + rhs.re_, // 실수부끼리 더함
        lhs.im_ + rhs.im_  // 허수부끼리 더함
    );
}

ComplexNonMember operator+(double lhs, const ComplexNonMember& rhs) {
    return ComplexNonMember(lhs) + rhs; // lhs를 lhs + 0i로
    바꿔 더함
}

void nonMemberExample() {
    ComplexNonMember z1(1.0, 2.0); // 1 + 2i
    ComplexNonMember z2(3.0, 4.0); // 3 + 4i

    ComplexNonMember z3 = z1 + z2;    // operator+(z1, z2)
    ComplexNonMember z4 = 1.0 + z1;   // operator+(1.0, z1)
}

```

멤버 함수 방식에서는 왼쪽 피연산자가 항상 현재 객체(`*this`)가 된다. 따라서 `a + b` 는 `a.operator+(b)` 처럼 해석된다. 구현이 간단하고 객체의 private 멤버에 접근하기 쉽지만, 왼쪽 피연산자가 사용자 정의 타입이 아닐 때는 사용할 수 없다.

비멤버 함수 방식에서는 두 피연산자를 모두 매개변수로 받는다. `double + Complex` 처럼 왼쪽 피연산자가 기본 타입인 경우에도 대응하기 쉽다. private 멤버 접근이 필요하다면 `friend` 로 선언한다.

Python에서는 `a + b` 가 먼저 `a.__add__(b)` 를 호출한다. 왼쪽 객체가 처리하지 못하면 오른쪽 객체의 reflected 메서드인 `b.__radd__(a)` 를 시도한다. C++에는 Python의 `__radd__` 와

완전히 같은 별도 프로토콜은 없다. Python은 왼쪽 객체가 실패한 뒤 오른쪽 객체에게 다시 기회를 주는 규칙이 있지만, C++은 컴파일 시점의 오버로드 해석으로 호출할 함수를 고른다.

따라서 C++에서 `__radd__`에 가장 가까운 대응 방식은 비멤버 연산자 오버로드를 양쪽 피연산자 순서에 맞게 제공하는 것이다. 예를 들어 `z1 + z2`는 `operator+(Complex, Complex)`가 처리하고, `1.0 + z1`은 `operator+(double, Complex)`가 처리한다. 즉, Python의 `__radd__`처럼 오른쪽 객체 안에 별도 메서드가 있는 것은 아니지만, 비멤버 함수와 암시적 변환을 이용해 같은 사용성을 만들 수 있다.

1-2. 복합 대입 연산자

연산	C++	Python
<code>+=</code>	<code>operator+=</code>	<code>__iadd__</code>
<code>-=</code>	<code>operator-=</code>	<code>__isub__</code>
<code>*=</code>	<code>operator*=</code>	<code>__imul__</code>
<code>/=</code>	<code>operator/=</code>	<code>__itruediv__</code>

C++에서 `operator+=`는 보통 현재 객체를 직접 수정한 뒤 `*this`를 참조로 반환한다.

```
Fraction& operator+=(const Fraction& rhs) {
    num_ = num_ * rhs.den_ + rhs.num_ * den_;
    den_ = den_ * rhs.den_;
    reduce();
    return *this;
}
```

`return *this;`에서 `*this`는 현재 수정된 객체 자신을 의미한다. 반환 타입을 `Fraction&`로 두고 `*this`를 반환하는 이유는 `a += b += c` 같은 체이닝을 가능하게 하고, 불필요한 복사를 줄이기 위해서다. 참조로 반환하면 연산 결과가 새 임시 객체가 아니라 실제로 수정된 왼쪽 객체를 가리키므로, C++의 기본 복합 대입 연산자와 같은 사용 방식을 제공할 수 있다.

C++에서 `operator+=`를 구현한다고 해서 `operator+`가 자동으로 생기지는 않는다. 두 연산자는 별도로 정의해야 한다. 다만 보통 `operator+` 내부에서 `operator+=`를 재사용해 중복을 줄인다.

Python에서는 `a += b`가 먼저 `a.__iadd__(b)`를 호출한다. `__iadd__`가 없거나 `NotImplemented`를 반환하면 `__add__` 또는 `__radd__`를 이용한 뒤 이름 `a`를 새 객체에 다시 바인딩한다. 따라서 Python의 `+=`는 객체가 mutable인지 immutable인지에 따라 실제 동작 방식이 달라질 수 있다.

```

# mutable 객체 예시: list
# list는 __iadd__가 자기 자신을 직접 수정한다.
a = [1, 2]
alias = a

a += [3]

print(a)          # [1, 2, 3]
print(alias)      # [1, 2, 3]
print(a is alias) # True

# immutable 객체 예시: tuple
# tuple은 내부를 수정할 수 없으므로 새 tuple을 만들고,
# 이름 a만 새 객체에 다시 바인딩된다.
b = (1, 2)
alias = b

b += (3,)

print(b)          # (1, 2, 3)
print(alias)      # (1, 2)
print(b is alias) # False

```

위 예시에서 `list`는 mutable이므로 `a += [3]` 이후에도 같은 객체가 유지된다. 그래서 같은 리스트를 가리키던 `alias`도 변경된 내용을 본다. 반대로 `tuple`은 immutable이므로 `b += (3,)`가 기존 tuple을 고치는 것이 아니라 새 tuple을 만든다. 따라서 `b`는 새 객체를 가리키지만, `alias`는 여전히 기존 tuple을 가리킨다.

1-3. 비교 연산자

연산	C++	Python
<code>==</code>	<code>operator==</code>	<code>__eq__</code>
<code>!=</code>	<code>operator!=</code>	<code>__ne__</code>
<code><</code>	<code>operator<</code>	<code>__lt__</code>
<code><=</code>	<code>operator<=</code>	<code>__le__</code>
<code>></code>	<code>operator></code>	<code>__gt__</code>
<code>>=</code>	<code>operator>=</code>	<code>__ge__</code>

C++20 이전에는 `operator==` 를 정의해도 `operator!=` 가 자동으로 생기지 않았다. 따라서 `!=` 까지 필요하면 직접 구현해야 했다. C++20부터는 비교 연산자 재작성 규칙이 도입되어, `operator==` 를 바탕으로 `!=` 후보를 만들 수 있다. 또한 `<=>` (three-way comparison)를 정의하면 여러 관계 연산을 더 일관적으로 생성할 수 있다.

Python에서는 `__eq__` 만 정의하고 `__ne__` 를 정의하지 않으면, 일반적으로 `!=` 는 `__eq__` 결과를 부정하는 방식으로 동작한다. 단, 비교 대상 타입을 처리할 수 없어 `NotImplemented` 가 관여하는 경우에는 양쪽 객체의 비교 메서드 탐색 규칙이 적용된다.

1-4. 단항 연산자

연산	C++	Python
단항 <code>-</code>	<code>operator-()</code>	<code>__neg__</code>
단항 <code>+</code>	<code>operator+()</code>	<code>__pos__</code>
<code>~</code>	<code>operator~()</code>	<code>__invert__</code>

C++에서 이항 `-` 와 단항 `-` 는 매개변수 개수로 구별된다. 멤버 함수 기준으로 단항 `operator-()` 는 매개변수가 없고, 이항 `operator-(const T& rhs)` 는 오른쪽 피연산자 하나를 받는다.

```
Fraction operator-() const; // 단항 -
Fraction operator-(const Fraction& rhs) const; // 이항 -
```

Python도 의미는 비슷하지만 메서드 이름이 명확히 분리되어 있다. `-a` 는 `a.__neg__()` 를 호출하고, `a - b` 는 `a.__sub__(b)` 를 호출한다.

1-5. 첨자 연산자

C++의 첨자 연산자는 `operator[]` 로 오버로딩한다.

```
int& operator[](int index);
int operator[](int index) const;
```

C++에서는 읽기와 쓰기가 하나의 `operator[]` 로 표현될 수 있다. Python처럼 `__getitem__` 과 `__setitem__` 에 해당하는 함수가 따로 호출되는 것이 아니다. C++의 핵심은 `operator[]` 가 무엇을 반환하느냐이다.

```
class Vector {
private:
    double data_[3];
```

```

public:
    double& operator[](int index) {
        return data_[index];    // 참조 반환: 대입 가능
    }

    const double& operator[](int index) const {
        return data_[index];    // const 참조 반환: 읽기만
        가능
    }
};

void example() {
    Vector v;

    v[0] = 3.14;    // 쓰기: non-const operator[]가 double&를 반환
    double x = v[0];    // 읽기: 반환된 double&에서 값을 읽음

    const Vector& cv = v;
    double y = cv[0];    // 읽기: const operator[] 호출

    // cv[0] = 2.71;
    // 오류: const operator[]는 const double&를 반환하므로 대입
    불가
}

```

따라서 C++은 읽기/쓰기 문맥에 따라 `operator[]` 와 다른 별도 함수를 선택하는 방식이 아니다. 비-`const` 객체에서는 `operator[]` 가 원소의 참조(`double&`)를 반환하므로 그 결과가 대입문의 왼쪽 값(lvalue)이 될 수 있고, `const` 객체에서는 `const` 버전이 호출되어 수정이 막힌다. 즉, C++은 반환 타입의 참조성(`T&`, `const T&`)과 객체의 `const` 여부로 읽기와 쓰기 가능성을 구분한다.

Python은 읽기와 쓰기 프로토콜을 분리한다.

```

def __getitem__(self, index): ...
def __setitem__(self, index, value): ...

```

`a[0]` 은 `__getitem__`, `a[0] = value` 는 `__setitem__` 을 호출한다. Python은 문맥을 기준으로 다른 특수 메서드를 호출하기 때문에 읽기와 쓰기의 역할이 더 명시적으로 분리된다.

1-6. 함수 호출 연산자

C++의 `operator()` 를 구현한 객체는 함수처럼 호출할 수 있다.

```
struct Adder {  
    int base;  
    int operator()(int x) const { return base + x; }  
};
```

이런 객체를 함수 객체(functor)라고 부르는 이유는 객체가 상태를 가질 수 있으면서도 함수 호출 문법으로 실행되기 때문이다. 일반 함수는 함수 내부에 별도의 객체 상태를 저장하기 어렵지만, 함수 객체는 멤버 변수를 가질 수 있다.

```
Adder add5{5};  
Adder add10{10};  
  
int a = add5(3);    // add5.operator()(3), 결과: 8  
int b = add10(3);   // add10.operator()(3), 결과: 13
```

위 예시에서 `add5` 와 `add10` 은 같은 `operator()` 를 사용하지만, 각 객체가 가진 `base` 값이 다르기 때문에 실행 결과가 달라진다. 즉, 함수 객체는 “함수처럼 호출되는 객체”이면서 동시에 “자기 상태를 보관하는 객체”다. 정렬 기준, 누적 계산기, 필터 조건처럼 상태가 필요한 동작을 함수처럼 전달할 때 유용하다.

람다(lambda)는 이름이 없는 짧은 함수를 그 자리에서 만들어 사용하는 문법이다. C++에서는 `[]` 캡처 목록, `()` 매개변수 목록, `{}` 함수 본문으로 람다를 작성한다. 특히 캡처 목록을 사용하면 람다 바깥에 있던 지역 변수를 람다 내부에 저장하거나 참조할 수 있다. C++ 람다도 내부적으로는 호출 연산자를 가진 익명 함수 객체에 가깝다.

```
int base = 5;  
auto add = [base](int x) {  
    return base + x;  
};  
  
int result = add(3); // 내부적으로 add.operator()(3)와 비슷하게 동작
```


위 람다는 `base` 를 캡처해서 보관하고, `add(3)` 처럼 함수 호출 문법으로 실행된다. 컴파일러는 이런 람다를 이름 없는 클래스 객체처럼 다루며, 그 클래스 안에 `operator()` 가 있다고 생각하면 이해하기 쉽다.

Python에서는 `__call__` 을 구현하면 객체를 함수처럼 호출할 수 있다.

```
class Adder:
    def __init__(self, base):
        self.base = base

    def __call__(self, x):
        return self.base + x
```

Python의 람다는 함수 객체를 생성한다. C++ 람다가 컴파일러가 생성한 클로저 객체인 것과 비교하면, 두 언어 모두 "상태를 가진 호출 가능한 객체"라는 개념을 공유한다.

1-7. 출력 연산자와 문자열 변환

C++에서 `std::ostream` 에 대한 출력 연산자는 일반적으로 비멤버 함수로 정의한다.

```
friend std::ostream& operator<<(std::ostream& os, const Fraction& f);
```

`cout << f` 에서 왼쪽 피연산자는 `std::ostream` 이다. `Fraction` 의 멤버 함수로 만들면 왼쪽 피연산자가 `Fraction` 이어야 하므로 자연스러운 출력 문법을 만들 수 없다. 따라서 `operator<<` 는 비멤버 함수로 구현한다.

반환 타입이 `std::ostream&` 인 이유는 체이닝 때문이다. `cout << a << b` 는 `(cout << a) << b` 처럼 이어지므로, 첫 번째 출력 결과가 다시 스트림 참조를 반환해야 한다.

Python에서는 `__str__` 와 `__repr__` 가 문자열 표현을 만든다. `__str__` 는 사용자가 읽기 쉬운 문자열을 만드는 데 주로 쓰이고, `__repr__` 는 디버깅이나 개발자 확인에 적합한 더 명확한 표현을 만드는 데 주로 쓰인다.

관점	C++ <code>operator<<</code>	Python <code>__str__</code> , <code>__repr__</code>
공통점	객체를 사람이 읽을 수 있는 형태로 표현	객체를 사람이 읽을 수 있는 문자열로 표현
출력 대상	<code>ostream</code> 인자로 결정	호출하는 쪽이 결정
결과	스트림에 직접 기록	문자열 반환
체이닝	<code>cout << a << b</code> 가능	해당 없음

관점	C++ <code>operator<<</code>	Python <code>__str__</code> , <code>__repr__</code>
대표 사용	<code>cout</code> , <code>cerr</code> , <code>fstream</code> 등	<code>str()</code> , <code>print()</code> , f-string, 대화형 출력 등
역할 구분	보통 <code>operator<<</code> 하나가 출력 형식을 담당	<code>__str__</code> 는 사용자용, <code>__repr__</code> 는 개발자/디버깅용 표현

공통점은 두 방식 모두 객체를 사람이 이해할 수 있는 형태로 표현하도록 사용자 정의 타입에 출력/표현 규칙을 붙인다는 점이다. 예를 들어 `Fraction(1, 2)` 를 `1/2` 처럼 보이게 만드는 역할을 한다.

차이점은 책임의 위치다. C++은 `operator<<` 가 `std::ostream& os` 를 직접 받아서 “어느 스트림에 어떤 내용을 쓸지”까지 처리한다. 그래서 같은 함수가 `cout`, `cerr`, 파일 스트림에 모두 사용할 수 있고, `std::ostream&` 를 반환하여 `cout << a << b` 같은 체이닝을 지원한다. 반면 Python의 `__str__` 와 `__repr__` 는 출력 대상에는 관여하지 않고, 객체가 “나는 어떤 문자열로 표현될 것인가”만 정한다. 그 문자열을 터미널에 출력할지, 파일에 쓸지, 로그에 남길지는 `print`, f-string, 파일 객체 등 호출하는 쪽이 결정한다.

즉, C++은 “스트림에 쓰는 행위”와 “객체 표현”이 `operator<<` 안에서 함께 다루어지고, Python은 “객체의 문자열 표현”과 “실제 출력 행위”가 분리되어 있다.

1-8. bool 변환 연산자

C++에서는 `operator bool()` 을 구현하여 객체의 truth value를 정의할 수 있다.

```
explicit operator bool() const {
    return num_ != 0;
}
```

`explicit` 을 붙이는 이유는 의도하지 않은 암시적 변환을 막기 위해서다. 예를 들어 bool 변환이 너무 자유롭게 허용되면 산술 연산이나 오버로드 선택에서 예기치 않은 변환이 발생할 수 있다.

Python에서는 `__bool__` 이 truth value를 정의한다. `if obj:` 문맥에서 `obj.__bool__()` 이 호출된다. `__bool__` 이 없으면 `__len__` 결과가 0인지 여부로 판단하는 규칙도 있다.

1-9. C++에만 있는 연산자 (Python 대응 없음)

다음 연산자들은 Python에 같은 의미의 연산자 오버로딩 대상이 없다. 여기서 “대응 없음”은 Python에 완전히 비슷한 기능이 전혀 없다는 뜻이 아니라, C++처럼 해당 기호를 연산자로 직접 오버로딩하는 방식이 없다는 뜻이다.

C++ 연산자	Python에 없는 이유
<code>++</code> , <code>--</code>	Python은 증가/감소 연산자를 별도로 두지 않는다. <code>x += 1</code> , <code>x -= 1</code> 처럼 대입과 산술 연산을 명시적으로 쓰도록 설계되었다. 또한 <code>int</code> , <code>float</code> , <code>str</code> , <code>tuple</code> 같은 기본 타입은 immutable이므로 값을 제자리에서 증가시키기보다 새 값을 만들어 이름에 다시 바인딩하는 모델과 잘 맞는다. 참고로 Python에서 <code>++x</code> 는 증가 연산이 아니라 단항 <code>+</code> 를 두 번 적용한 표현으로 해석된다.
<code>-></code>	C++의 <code>-></code> 는 포인터가 가리키는 객체의 멤버에 접근하기 위한 연산자다. Python에는 C++식 포인터 노출과 역참조 문법이 없고, 모든 객체의 속성 접근을 <code>obj.attr</code> 형태의 <code>.</code> 연산으로 통일한다. 즉, Python은 “객체 참조”를 언어 수준에서 자동으로 다루기 때문에 별도의 포인터 멤버 접근 연산자가 필요하지 않다.
<code>&&</code> , <code>\\ \\ </code>	Python은 논리 연산에 <code>and</code> , <code>or</code> 키워드를 사용한다. 이들은 단락 평가 (short-circuit)를 수행하므로 왼쪽 값만 보고 오른쪽 표현식의 평가 여부를 결정할 수 있어야 한다. 일반적인 연산자 오버로딩 함수로 만들면 함수 호출 전에 피연산자가 먼저 평가되어 단락 평가 의미가 깨질 수 있다. 그래서 Python은 <code>and</code> , <code>or</code> 를 사용자 정의 오버로딩 대상으로 제공하지 않는다. 대신 객체의 참/거짓 판단은 <code>__bool__</code> 또는 <code>__len__</code> 으로 정의한다.
<code>new</code> , <code>delete</code>	C++은 객체 생성과 메모리 해제를 개발자가 직접 제어할 수 있으므로 <code>new</code> , <code>delete</code> 연산자가 존재한다. Python은 객체 생성을 <code>ClassName(...)</code> 호출로 표현하고, 메모리 관리는 참조 카운팅과 가비지 컬렉션이 담당한다. Python에도 <code>__new__</code> 메서드는 있지만 이는 객체 생성 과정에 개입하는 특수 메서드일 뿐, C++의 <code>new</code> 처럼 메모리를 직접 할당하는 연산자는 아니다. 또한 <code>del</code> 은 이름이나 속성의 바인딩을 제거하는 문법이지 C++의 <code>delete</code> 처럼 객체 메모리를 즉시 해제하는 연산자가 아니다.

2. 구현 코드

`Fraction` 클래스는 두 언어에서 모두 다음 원칙으로 구현했다.

- 분모가 0이면 예외를 발생시킨다.
- 모든 객체는 생성과 연산 후 항상 기약분수 상태를 유지한다.
- 분모는 항상 양수로 저장한다.
- 산술 연산, 복합 대입, 비교, bool 변환, 첨자 접근, 문자열 출력을 제공한다.
- 전체 코드는 Appendix A와 Appendix B에 첨부한다.

3. 실행 결과

3-1. C++ 실행 결과

```

1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
1/2 == 1/2 : true
1/2 is nonzero
a += b : 5/6
분자: 5, 분모: 6

```

3-2. Python 실행 결과

```

1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
1/2 == 1/2 : True
1/2 is nonzero
a += b : 5/6
분자: 5, 분모: 6

```

C++은 `std::boolalpha` 를 사용하면 bool 값이 `true`, `false` 로 출력된다. Python은 bool 객체의 문자열 표현이 `True`, `False` 이므로 대문자로 출력된다.

4. 분석 및 결론

4-1. gcd 라이브러리 사용 분석

`Fraction` 클래스에서 가장 중요한 내부 규칙은 모든 분수를 항상 기약분수 형태로 유지하는 것이다. 이를 위해 C++ 구현에서는 `<numeric>` 헤더의 `std::gcd` 를 사용했다.

```

#include<numeric>

int divisor = std::gcd(num_ < 0 ? -num_ : num_, den_);
num_ /= divisor;
den_ /= divisor;

```

`std::gcd` 는 두 정수의 최대공약수(greatest common divisor)를 구하는 표준 라이브러리 함수다. 예를 들어 `std::gcd(10, 15)` 는 5 를 반환한다. 따라서 `10/15` 는 분자와 분모를 모두 5 로 나누어 `2/3` 으로 약분할 수 있다.

이 보고서의 C++ 코드에서는 직접 유클리드 호제법을 구현하지 않고 `std::gcd` 를 사용했다. 그 이유는 표준 라이브러리를 쓰면 코드가 짧아지고, 의도가 명확해지며, 검증된 구현을 사용할 수 있기 때문이다. 또한 `reduce()` 함수 안에서 `std::gcd` 를 호출하도록 만들면 생성자, `operator+`, `operator-`, `operator*`, `operator/`, `operator+=` 의 결과가 모두 같은 방식으로 정규화된다.

Python 구현에서도 같은 목적을 위해 `math.gcd` 를 사용했다.

```
from math import gcd

divisor = gcd(abs(self.num), self.den)
self.num //= divisor
self.den //= divisor
```

두 언어 모두 `gcd` 라이브러리를 사용함으로써 분수의 핵심 불변식인 “분모는 양수이고, 분자와 분모는 서로소인 상태”를 쉽게 유지할 수 있다.

4-2. `operator<<` 와 `__str__` 의 설계 철학 차이

C++의 `operator<<` 와 Python의 `__str__` 는 모두 객체의 출력 형식을 정의한다는 공통점이 있다. 그러나 두 기능의 중심은 다르다.

C++의 `operator<<` 는 출력 대상인 `std::ostream` 과 객체를 함께 다룬다. 따라서 같은 객체도 `cout`, `cerr`, 파일 스트림 등 다양한 출력 대상으로 보낼 수 있고, 스트림 참조를 반환해 체이닝을 지원한다. C++의 접근 방식은 연산자 표현과 입출력 스트림 모델을 결합한 형태다.

Python의 `__str__` 는 객체가 자기 자신을 문자열로 표현하는 방법을 제공한다. 실제 출력 대상은 `print`, 파일 쓰기, 로깅, f-string 등 호출하는 쪽이 결정한다. Python의 접근 방식은 객체 표현과 출력 행위를 분리하는 형태다.

4-3. `operator[]` 와 `__getitem__`, `__setitem__` 의 차이

C++의 `operator[]` 는 하나의 연산자 함수로 읽기와 쓰기를 모두 처리할 수 있다. 핵심은 반환 타입이다. 비-`const` 객체의 `operator[]` 가 참조를 반환하면 대입문의 왼쪽 값(lvalue)이 될 수 있다. 반대로 `const` 객체에서는 `const` 버전이 호출되어 수정이 제한된다.

Python은 읽기와 쓰기를 서로 다른 특수 메서드로 분리한다. `obj[index]` 는 `__getitem__`, `obj[index] = value` 는 `__setitem__` 이 담당한다. 이 방식은 문맥별 역할이 명확하지만, C++처럼 반환 참조를 통해 쓰기 가능성을 표현하지는 않는다.

4-4. `Fraction` 구현에서 느낀 차이점

가장 큰 차이는 “객체를 얼마나 명시적으로 통제해야 하는가”였다. C++은 참조 반환, `const`, `friend`, `explicit`, 값 복사 여부 등 타입과 메모리 관점의 결정을 세밀하게 내려야 한다. 그만큼 코드가 길어지지만, 컴파일 시점에 많은 규칙을 확인할 수 있고 성능과 인터페이스를 정교하게 설계할 수 있다.

Python은 같은 기능을 더 짧고 직접적으로 표현할 수 있다. `__add__`, `__bool__`, `__getitem__` 처럼 프로토콜 이름만 맞추면 언어 문법과 자연스럽게 연결된다. 대신 타입 오류

나 지원하지 않는 연산 처리는 런타임에 드러나므로, `NotImplemented` 반환이나 예외 처리 같은 방식을 신경 써야 한다.

결론적으로 C++의 연산자 오버로딩은 타입 안정성과 제어권을 중시하고, Python의 특수 메서드는 객체 프로토콜의 일관성과 표현력을 중시한다. `Fraction` 처럼 수학적 의미가 분명한 타입에서는 두 언어 모두 연산자 오버로딩이 코드 가독성을 높여 준다. 다만 연산자의 의미가 직관적이지 않은 타입에 남용하면 오히려 읽기 어려운 코드가 될 수 있다.

Appendix A. C++ 전체 코드

```
#include<iostream>
#include<numeric>
#include<stdexcept>

class Fraction {
public:
    Fraction(int numerator, int denominator)
        : num_(numerator), den_(denominator) {
        if (denominator == 0) {
            throw std::invalid_argument("denominator must not be zero");
        }
        reduce();
    }

    Fraction operator+(const Fraction& rhs) const {
        return Fraction(num_ * rhs.den_ + rhs.num_ * den_,
            den_ * rhs.den_);
    }

    Fraction operator-(const Fraction& rhs) const {
        return Fraction(num_ * rhs.den_ - rhs.num_ * den_,
            den_ * rhs.den_);
    }

    Fraction operator*(const Fraction& rhs) const {
        return Fraction(num_ * rhs.num_, den_ * rhs.den_);
    }
}
```

```

Fraction operator/(const Fraction& rhs) const {
    if (rhs.num_ == 0) {
        throw std::invalid_argument("division by zero fraction");
    }
    return Fraction(num_ * rhs.den_, den_ * rhs.num_);
}

Fraction operator-() const {
    return Fraction(-num_, den_);
}

Fraction& operator+=(const Fraction& rhs) {
    num_ = num_ * rhs.den_ + rhs.num_ * den_;
    den_ = den_ * rhs.den_;
    reduce();
    return *this;
}

bool operator==(const Fraction& rhs) const {
    return num_ == rhs.num_ && den_ == rhs.den_;
}

bool operator!=(const Fraction& rhs) const {
    return !(*this == rhs);
}

bool operator<(const Fraction& rhs) const {
    return num_ * rhs.den_ < rhs.num_ * den_;
}

explicit operator bool() const {
    return num_ != 0;
}

int operator[](int index) const {
    if (index == 0) {
        return num_;
    }
}

```

```

    }
    if (index == 1) {
        return den_;
    }
    throw std::out_of_range("Fraction index must be 0 or 1");
}

friend std::ostream& operator<<(std::ostream& os, const
Fraction& f) {
    if (f.den_ == 1) {
        return os << f.num_;
    }
    return os << f.num_ << "/" << f.den_;
}

private:
    int num_;
    int den_;

    void reduce() {
        if (den_ < 0) {
            num_ = -num_;
            den_ = -den_;
        }

        int divisor = std::gcd(num_ < 0 ? -num_ : num_, den_);

        num_ /= divisor;
        den_ /= divisor;
    }
};

int main() {
    Fraction a(1, 2);
    Fraction b(1, 3);

    std::cout << a << " + " << b << " = " << (a + b) << st

```



```

d::endl;
    std::cout << a << " * " << b << " = " << (a * b) << st
d::endl;
    std::cout << a << " == " << Fraction(2, 4) << " : "
        << std::boolalpha << (a == Fraction(2, 4)) <<
std::endl;

    if (a) {
        std::cout << a << " is nonzero" << std::endl;
    }

    a += b;
    std::cout << "a += b : " << a << std::endl;
    std::cout << "분자: " << a[0] << ", 분모: " << a[1] << s
td::endl;
}

```

Appendix B. Python 전체 코드

```

from math import gcd

class Fraction:
    def __init__(self, numerator: int, denominator: int):
        if denominator == 0:
            raise ValueError("denominator must not be zero")

        self.num = numerator
        self.den = denominator
        self._reduce()

    def _reduce(self):
        if self.den < 0:
            self.num = -self.num
            self.den = -self.den

        divisor = gcd(abs(self.num), self.den)

```

```

        self.num //= divisor
        self.den //= divisor

    def __coerce__(self, other):
        if isinstance(other, Fraction):
            return other
        return NotImplemented

    def __add__(self, other):
        other = self._coerce(other)
        if other is NotImplemented:
            return NotImplemented
        return Fraction(self.num * other.den + other.num *
self.den, self.den * other.den)

    def __sub__(self, other):
        other = self._coerce(other)
        if other is NotImplemented:
            return NotImplemented
        return Fraction(self.num * other.den - other.num *
self.den, self.den * other.den)

    def __mul__(self, other):
        other = self._coerce(other)
        if other is NotImplemented:
            return NotImplemented
        return Fraction(self.num * other.num, self.den * ot
her.den)

    def __truediv__(self, other):
        other = self._coerce(other)
        if other is NotImplemented:
            return NotImplemented
        if other.num == 0:
            raise ZeroDivisionError("division by zero fract
ion")
        return Fraction(self.num * other.den, self.den * ot
her.num)

```

```

def __neg__(self):
    return Fraction(-self.num, self.den)

def __iadd__(self, other):
    other = self._coerce(other)
    if other is NotImplemented:
        return NotImplemented

    self.num = self.num * other.den + other.num * self.
den
    self.den = self.den * other.den
    self._reduce()
    return self

def __eq__(self, other):
    other = self._coerce(other)
    if other is NotImplemented:
        return NotImplemented
    return self.num == other.num and self.den == other.
den

def __ne__(self, other):
    result = self.__eq__(other)
    if result is NotImplemented:
        return NotImplemented
    return not result

def __lt__(self, other):
    other = self._coerce(other)
    if other is NotImplemented:
        return NotImplemented
    return self.num * other.den < other.num * self.den

def __bool__(self):
    return self.num != 0

def __getitem__(self, index):

```

```

    if index == 0:
        return self.num
    if index == 1:
        return self.den
    raise IndexError("Fraction index must be 0 or 1")

def __str__(self):
    if self.den == 1:
        return str(self.num)
    return f"{self.num}/{self.den}"

def __repr__(self):
    return f"Fraction({self.num},{self.den})"

a = Fraction(1, 2)
b = Fraction(1, 3)

print(f"{a} +{b} ={a + b}")
print(f"{a} *{b} ={a * b}")
print(f"{a} =={Fraction(2, 4)} :{a == Fraction(2, 4)}")

if a:
    print(f"{a} is nonzero")

a += b
print(f"a += b :{a}")
print(f"분자:{a[0]}, 분모:{a[1]}")

```